UPDATED IMPLEMENTATION PLAN • v2.1

Context-Aware, Self-Verifying Natural Language to MiniLang Compiler

A phase-by-phase, step-by-step build plan for a hybrid NLP + compiler project, with a defined, implementable and measurable novelty package.

What changed from v1	Why
Novelty is now designed into the architecture, not bolted on at the end	Clarification and context handling were marked 'post-MVP' in v1, which leaves nothing to show for the 4 novelty marks
Round-trip AST check demoted to a supporting feature	With deterministic AST-based codegen it can almost never fail, so it is weak as the headline claim (explained in Section 1)
New headline novelty: compiler-informed interpretation + two-way verification	The compiler's symbol table actively resolves English ambiguity, and back-translation checks the NLP's interpretation, which code round-trip cannot
Evaluation now includes ablation and paraphrase-consistency metrics	A novelty claim backed by numbers is far easier to defend than a feature list

Sources merged: the original 12-page blueprint (compiler_proj.pdf), the novelty notes, and the project brief. Everything in v1 that was sound (MVP-first, rule-based NLP, fixed language, phase gates, CLAUDE.md rules) is kept.

1. Honest review of the current plan

The v1 blueprint is solid engineering: fixed language first, deterministic NLP, clean module boundaries, tests per phase. Keep all of that. The weak point is novelty, and the novelty notes partly fix it but lean on the wrong feature.

What holds up

- Rule-based NLP for the MVP. Correct call: reproducible, offline, testable, and an LLM would make the 'compiler' part look decorative.
- Intermediate representation (IR) between NLP and compiler. This is the backbone that makes every novelty feature below possible.
- Phase gates with exit tests. Keep these strictly; they are what stop an 8-week project from ending half-built.

What is weak, and the fix

Problem 1 - Round-trip AST verification is a weak headline. In this design the code generator prints MiniLang *from the AST*, and the parser reads the *same grammar* back. If both are correct, the re-parsed AST always equals the original. The check can only fail when the printer or parser has a bug, which golden tests already catch. The notes admit this by needing a 'deliberate corruption hook' to ever show it failing. An examiner can reasonably ask: *'When would this fail in real use?'* and there is no good answer. It also cannot detect the most likely real error: the NLP misunderstanding the English.

Fix: keep it, but as one layer of a three-layer verifier, and make the headline the parts that catch *interpretation* errors (back-translation and context-aware disambiguation).

Problem 2 - v1 postpones the novel parts. v1's risk table says to add clarification questions 'only after the MVP'. If that slips (it usually does), the final demo is a plain English-to-code converter, which you already know is not novel.

Fix: the IR and the NLP output are designed from Phase 2 to carry multiple candidate interpretations and confidence. Clarification then costs little extra work later.

Problem 3 - a real ambiguity is hidden in v1's own examples. 'print hello world' becomes `PRINT "hello world";` but 'print age' becomes `PRINT age;`. The only way to know *age* is a variable is to know it was declared. That is a compiler fact (symbol table), not an NLP fact. v1 never says how this is decided.

Fix: this gap is the seed of the strongest novelty: let the compiler's symbol table inform the NLP.

Facts vs. inference about novelty

- **Fact:** NL-to-code and NL-to-DSL systems exist, and clarification questions, back-translation and round-trip testing each exist individually in research and tools. Do not claim any single one is new.
- **Reasonable claim:** the specific combination - a compiler symbol table feeding back into English interpretation, explicit ambiguity handling, and layered verification inside a formally specified mini-language, with measured results - is a defensible *project-level* contribution.
- **Uncertain:** how your examiners weight 'novel' vs 'well-executed'. A short related-work paragraph (3-5 citations) protects you either way; do it in Week 1.

2. The novelty package

Five features. N1-N3 are **core** (must ship). N4 is **core-light** (small effort). N5 is the **evidence** that turns features into a claim. Each one is deterministic, needs no paid API, and fits a Python standard-library build.

N1. Compiler-informed interpretation (headline feature)

In a normal pipeline, information flows one way: NLP -> compiler. Here, the compiler's **symbol table and statement history flow back** into the NLP stage to resolve what English leaves open. This is the feature that makes the project genuinely hybrid rather than two systems glued together.

English problem	How the symbol table resolves it	Example
Variable or text?	Fixed policy: exact match to a declared variable -> identifier; quoted text -> always string; anything else -> text by default; a likely typo of a declared name -> confirm (see Typos row).	after *LET age = 20*: 'print age' -> PRINT age; 'print adult' -> PRINT "adult"; 'print total' -> PRINT "total";
Pronouns / references	'it', 'that', 'the variable' resolve to the most recently assigned or referenced variable, tracked by a small context stack.	'set score to 10. add 5 to it.' -> score = score + 5;
Implicit declaration	'set x to 5' when x is undeclared: generate LET instead of assignment (with a note), rather than failing.	x undeclared -> LET x = 5; (warning shown)
Typos in names	Edit distance against declared names; never silently auto-correct, always confirm.	'print scroe' -> "Did you mean variable score?"
Type-aware wording	Symbol table stores inferred types (number/string); reject or flag 'add 5 to name' when name is text.	Type error with plain-English explanation

Why it matters for marks: it is a concrete, demonstrable interaction between a classical compiler data structure and NLP. You can measure it with an ablation: run the same test set with context ON vs OFF and report the accuracy difference (N5).

N2. Ambiguity detection and clarification dialogue

The NLP stage returns a **list of candidate interpretations**, each with a score, instead of one answer. A decision rule then picks one of three outcomes:

Outcome	Rule (tune thresholds in Phase 6)	Example
Accept	Exactly one candidate, or top score clearly ahead of the next (margin above threshold), and no missing required slot.	'set age to 21'
Clarify	Two or more plausible candidates, or a required slot is missing. Ask a targeted question listing the options.	'increase x' -> "By how much?" 'set x to 10 or 20' -> "10 or 20?"
Reject	No candidate matches a supported pattern. Hand over to N4 for a suggestion.	'sort the list descending'

Ambiguity sources to cover explicitly: missing values, alternatives ('or'), likely typos of declared names, unclear operator scope ('multiply x by 2 plus 3' -> (x*2)+3 or x*(2+3)?), and unclear block boundaries in IF/REPEAT. In non-interactive (file) mode, clarification becomes a diagnostic so batch runs never hang.

N3. Two-way, three-layer verification

Every accepted statement passes three checks. Each catches a different class of error; together they answer the examiner's 'how do you know the output is right?'

Layer	What it does	Catches	Cannot catch
V1 Back-translation (core)	Generate a canonical English sentence from the AST with fixed templates and show it: "I understood: create variable age with value 20." It is always displayed. The user is asked to confirm only when the decision is CLARIFY, or for every statement in strict mode (--confirm).	NLP misinterpretation - the error round-trip AST cannot see	Errors the user does not notice (it supports a check, it is not a proof)

Layer	What it does	Catches	Cannot catch
V2 Structural round-trip (supporting)	Re-parse generated MiniLang with the standalone parser; compare to the original AST.	Code generator / printer bugs, grammar drift	Interpretation errors
V3 Execution equivalence (core-light)	Run the AST with a tiny tree-walking evaluator, and run the generated MiniLang through the parser + interpreter; compare final variable states and printed output.	Semantic bugs where code parses but means something else (e.g. wrong operator precedence)	Same bug in both evaluators (keep them independent)

Key point to say in the viva: V2 alone only checks the compiler against itself. V1 checks the compiler against the *user's intent*, and V3 checks *meaning*, not just shape. That layering is the defensible version of the round-trip idea. Describe it as verification against defined checks, never as proof that every English interpretation is correct.

N4. 'Did you mean' recovery for unsupported input

When input is rejected, find the closest supported pattern (token overlap + edit distance over the pattern templates in LANGUAGE_SPEC.md) and suggest a rewrite: *'store 10 in x'* is unsupported -> *"Try: set x to 10"*. Small effort, very visible in a demo, and it turns rejection into guidance.

N5. Measured evaluation (the evidence)

Covered in Section 8. Without numbers, N1-N4 are a feature list; with an ablation and paraphrase-consistency results, they are a contribution.

3. Updated architecture and data flow

Novel stages are marked [N]. Arrows going up are the feedback paths that make N1 work.

```
English sentence
      |
[A] Normalizer ............. keywords matched in lowercase; identifier case kept
      |
[B] Candidate generator ... pattern rules -> 0..k candidate intents, each with slots + score
      |            ^
      |            |  [N1] Context resolver: symbol table + reference stack
      |            |       (variable vs string, 'it', typos, implicit declare)
      v            |
[C] Ambiguity gate [N2] -- accept --> continue
      |         \-- clarify --> ask user / emit diagnostic --> re-enter at [B]
      |          \-- reject  --> [N4] nearest-pattern suggestion
      v
[D] IR / AST builder (typed nodes)
      |
[E] Semantic analyzer + symbol table  ----------------------------^ (feeds N1)
      |
[F] Code generator (AST -> MiniLang text)
      |
[G] Verifier [N3]:  V1 back-translation | V2 re-parse + AST compare | V3 execution equivalence
      |
[H] Trace / output:  input, normalized text, candidates + scores, decision,
                     AST, checks, back-translation, MiniLang, verification result
```

Data contracts (freeze these in Phase 1)

```
Candidate   = { intent: str, slots: dict, score: float, rule_id: str, notes: [str] }
NLPResult   = { status: "ok" | "ambiguous" | "unsupported",
                candidates: [Candidate], question: str | None, suggestion: str | None }
Diagnostic  = { level: "error" | "warning" | "info", stage: str, code: str, message: str }
Verification= { back_translation: str, v2_match: bool, v3_match: bool | None, details: [str] }
Session     = { symbol_table: {name: {type, declared_at}}, ref_stack: [name], history: [Stmt] }
```

Design rules

- No stage silently guesses. It returns a testable result or a diagnostic. (Kept from v1.)
- The NLP never emits MiniLang text. It emits candidates; only the code generator writes code.
- The symbol table has one owner (semantic analyzer). The context resolver reads it but never writes to it.
- Back-translation templates live next to codegen templates, one per AST node type, so adding a node forces both.

4. MiniLang specification (lock in Week 1)

Unchanged from v1 except: added >=, <=, != in conditions (cheap and more realistic), defined types, and defined the variable-vs-string rule.

```
program      := statement*
statement    := declaration | assignment | print_stmt | if_stmt | repeat_stmt
declaration  := "LET" IDENT "=" expression ";"
assignment   := IDENT "=" expression ";"
print_stmt   := "PRINT" expression ";"
if_stmt      := "IF" condition "THEN" block ("ELSE" block)?
repeat_stmt  := "REPEAT" NUMBER "TIMES" block
block        := "{" statement* "}"
condition    := expression ( ">" | "<" | "==" | ">=" | "<=" | "!=" ) expression
expression   := term   (("+" | "-") term)*
term         := factor (("*" | "/") factor)*
factor       := NUMBER | STRING | IDENT | "(" expression ")"
```

Rule	Decision
Types	number (integer/decimal) and string. No booleans as values in MVP; conditions only appear inside IF.
String + number	Semantic error in MVP (no implicit conversion). String + string is allowed (concatenation).
Identifiers	Letters, digits, underscore; must start with a letter; case-sensitive; MiniLang keywords reserved.
Normalization vs case	The normalizer lowercases only English command words and synonyms for matching. Extracted identifiers and quoted text keep their original spelling (totalAmount stays totalAmount).
Variable vs string in English	1) Exact match to a declared variable -> identifier. 2) Quoted text -> always a string. 3) Anything else -> string literal by default. 4) Within edit distance 2 of a declared name -> ask to confirm ('Did you mean score?'). No other case asks.
Nesting	IF and REPEAT blocks may contain one statement in MVP English input; multi-statement blocks only via 'then ... and ...' patterns if time allows.

Supported English patterns (starting set, expand only with spec + tests)

Intent	Example English	MiniLang
declare	make a variable called age and set it to 20	LET age = 20;
assign	set age to age plus 1 / change age to 30	age = age + 1;
increment / decrement	increase count by 2 / decrease count by 1	count = count + 2;
arith-assign	multiply total by 3 / divide total by 2	total = total * 3;
print	print age / show "Hello" / display hello world	PRINT age; PRINT "hello world";
if	if age is greater than 17 then print adult else print minor	IF age > 17 THEN { PRINT "adult"; } ELSE { ... }
repeat	repeat 3 times print hello	REPEAT 3 TIMES { PRINT "hello"; }
reference [N1]	add 5 to it (after a statement using score)	score = score + 5;

5. Repository structure

```
compiler-project/
|-- README.md  CLAUDE.md  DEVELOPMENT.md  PROJECT_OVERVIEW.md
|-- LANGUAGE_SPEC.md  ARCHITECTURE.md  TESTING.md  NOVELTY.md        <- new
|-- requirements.txt  pyproject.toml  .gitignore
|-- examples/ valid/  invalid/  ambiguous/                          <- new
|-- src/nlmini/
|   |-- cli.py  pipeline.py  diagnostics.py  session.py             <- session new
|   |-- nlp/
|   |   |-- normalizer.py
|   |   |-- patterns.py          # pattern table: rule_id, regex/tokens, intent, slots
|   |   |-- candidates.py        # produces scored candidates
|   |   |-- context.py           # [N1] symbol-table-aware resolution, ref stack, typos
|   |   |-- ambiguity.py         # [N2] accept / clarify / reject + questions
|   |   `-- suggest.py           # [N4] nearest supported pattern
|   |-- compiler/
|   |   |-- ast_nodes.py  ir_builder.py  lexer.py  parser.py
|   |   |-- semantic.py   symbols.py     codegen.py
|   |-- verify/
|   |   |-- backtranslate.py     # [N3-V1] AST -> canonical English
|   |   |-- roundtrip.py         # [N3-V2] re-parse + AST compare
|   |   |-- ast_eval.py          # [N3-V3] evaluator over AST
|   |   `-- interpreter.py       # [N3-V3] MiniLang interpreter (via parser)
|   `-- eval/
|       |-- dataset.py  metrics.py  run_eval.py                      # [N5]
`-- tests/ unit/  integration/  fixtures/  eval_data/
```

Documentation order (from v1, plus NOVELTY.md)

- PROJECT_OVERVIEW.md - problem, goals, non-goals, success criteria.
- LANGUAGE_SPEC.md - grammar, types, English patterns, rejected patterns, ambiguity rules.
- ARCHITECTURE.md - the Section 3 diagram and data contracts.
- NOVELTY.md - N1-N5, related work (3-5 citations), what is and is not claimed, metrics.
- DEVELOPMENT.md and CLAUDE.md - phase gates and coding-agent rules (Section 10).
- requirements.txt (pytest only), pyproject.toml, .gitignore; confirm a clean install runs the empty test suite.

6. Phase-by-phase, step-by-step implementation

Eight weeks, ten phases. Novelty work is spread through the plan, not left to the end. If your deadline is shorter, compress Phases 3-5, never Phases 6-8: those carry the novelty marks.

Phase	Timing	Focus	Novelty
0	Days 1-2	Setup, repo, docs skeleton	-
1	Week 1	Language spec, data contracts, fixtures, related work	Ambiguous fixtures; NOVELTY.md
2	Week 2	NLP front end with scored candidates	Multi-candidate output (enables N2)
3	Week 3	AST, IR builder, lexer, parser	Parser reused by V2/V3
4	Week 4	Semantic analysis, symbol table, session	Symbol table exposed for N1
5	Week 5	Code generation + back-translation + end-to-end CLI	N3-V1
6	Week 6 (first half)	Context resolver + ambiguity gate + suggestions	N1, N2, N4
7	Week 6 (second half) - Week 7	Verifier: round-trip + interpreter + execution equivalence	N3-V2, V3
8	Week 7	Evaluation dataset, metrics, ablation	N5
9	Week 8	Hardening, docs, report, demo	Novelty demo script

Phase 0 - Setup (Days 1-2)

Goal: A clean repository that installs and runs an empty test suite from the README alone.

Steps

1. Create the Git repo and the folder structure from Section 5 (empty modules with docstrings).
2. Create a virtual environment; requirements.txt with only `pytest>=8,<9`; pyproject.toml with package metadata and pytest config.
3. Write a placeholder `cli.py` that echoes input and exits 0.
4. Add one trivial passing test; commit.
5. Create empty doc files with headings so later phases fill them rather than invent structure.

Deliverable	Exit test (do not move on until this passes)
Runnable skeleton, docs headings, first commit	Fresh clone -> install -> `python -m pytest -q` passes; CLI starts

Phase 1 - Lock the language and contracts (Week 1)

Goal: Every later component has a fixed target: grammar, types, English patterns, data contracts and fixtures.

> **Novelty work in this phase:** Ambiguous fixtures and NOVELTY.md are written now so the target is fixed before code exists.

Steps

1. Write LANGUAGE_SPEC.md from Section 4: grammar, types, identifier rules, error behavior.
2. List supported English patterns per intent (3-5 phrasings each) and explicitly rejected patterns.
3. Write the ambiguity rules: which situations clarify, which reject, which auto-resolve (variable vs string, missing slot, 'or', operator scope).
4. Freeze the data contracts from Section 3 in ARCHITECTURE.md.
5. Create fixtures: at least 20 valid (English + expected MiniLang), 10 invalid, 10 ambiguous (English + expected question).

6. Write NOVELTY.md: N1-N5, plus 3-5 related-work citations (NL-to-code, NL-to-DSL, clarification in semantic parsing) and an explicit 'what we do not claim' list.

Deliverable	Exit test (do not move on until this passes)
Frozen spec, contracts, 40+ fixtures, NOVELTY.md draft	Team agrees spec is frozen; fixtures saved in tests/fixtures and examples/

Phase 2 - NLP front end (scored candidates) (Week 2)

Goal: Turn normalized English into zero or more scored candidate intents.

> **Novelty work in this phase:** Returning a ranked list instead of one answer is what makes N2 (clarification) and N5 (ambiguity metrics) possible later.

Steps

1. Normalizer: keep the original text; build a lowercased copy used only for matching command words and synonyms, and extract identifiers and quoted text from the original so their case is preserved; collapse spaces, strip trailing punctuation, map synonyms ('make/create/declare', 'show/display/print', 'plus/add', 'greater than/more than/above').
2. Pattern table in patterns.py: each rule has an id, a token/regex pattern, an intent, and slot extractors. Keep it data-driven so tests and N4 can read it.
3. Slot extraction: identifiers, numbers (including 'ten' to 10 for 0-20 if cheap), quoted strings, operators, condition and body clauses.
4. Candidate generator: run all rules, return every match with a score (e.g. fraction of tokens covered + rule specificity). Do not pick one yet.
5. Return status 'unsupported' when nothing matches. No exceptions escape.
6. Unit tests: every Phase 1 valid fixture yields the expected top candidate; invalid fixtures yield 'unsupported'.

Deliverable	Exit test (do not move on until this passes)
normalizer.py, patterns.py, candidates.py with tests	All valid fixtures map to the right intent + slots; invalid ones fail cleanly

Phase 3 - AST, IR builder, lexer, parser (Week 3)

Goal: Typed program representation, and an independent MiniLang parser for verification.

> **Novelty work in this phase:** This parser is reused by V2 and V3. Writing it independently of the code generator is what keeps the verification meaningful.

Steps

1. Define AST node dataclasses: Program, Let, Assign, Print, If, Repeat, BinOp, Compare, Num, Str, Var. Make them frozen and comparable (dataclass eq) so V2 is a simple equality check.
2. IR builder: Candidate -> AST nodes. One function per intent.
3. Lexer: tokens for keywords, identifiers, numbers, strings, operators, braces, semicolons; report line/column on errors.
4. Recursive-descent parser following the Section 4 grammar with correct precedence.
5. Tests: parse every expected-MiniLang fixture to an AST; malformed MiniLang returns errors, never crashes.

Deliverable	Exit test (do not move on until this passes)
ast_nodes.py, ir_builder.py, lexer.py, parser.py with tests	Every valid fixture: English -> AST matches parse(expected MiniLang)

Phase 4 - Semantic analysis, symbol table, session (Week 4)

Goal: Catch meaning errors and keep program state across sentences.

> **Novelty work in this phase:** The read-only symbol table view and reference stack are the inputs N1 needs.

Steps

1. symbols.py: symbol table storing name, inferred type, declaring statement.
2. semantic.py: undefined variable, redeclaration (warning), type mismatch (string arithmetic), division by literal zero, invalid REPEAT count.
3. Diagnostics in plain English with stage and code, e.g. E201 'score is used before it is created'.
4. session.py: Session object holding symbol table, reference stack (last referenced names) and statement history across a multi-sentence program.
5. Expose a read-only view of the symbol table for the NLP context resolver (Phase 6).
6. Positive and negative semantic tests.

Deliverable	Exit test (do not move on until this passes)
semantic.py, symbols.py, session.py with tests	All semantic fixtures produce expected diagnostics; valid programs produce none

Phase 5 - Code generation, back-translation, end-to-end CLI (Week 5)

Goal: A working English -> MiniLang pipeline with the first verification layer.

> **Novelty work in this phase:** N3-V1 back-translation ships here, early, so the demo always shows the system explaining what it understood.

Steps

1. codegen.py: one template per AST node; consistent spacing, semicolons, braces, string escaping, parentheses only where precedence needs them.
2. backtranslate.py (V1): one English template per AST node, e.g. Assign(x, BinOp(+, Var x, Num 5)) -> 'increase x by 5'. Keep wording canonical and unambiguous.
3. pipeline.py: normalize -> candidates (take top for now) -> AST -> semantic -> codegen -> back-translate.
4. CLI: single sentence, file mode (one sentence per line, shared session), `--trace`, `--output`, interactive REPL mode, and `--confirm` (strict mode: ask the user to confirm every back-translation).
5. Golden tests: generated MiniLang equals expected text for every valid fixture.

Deliverable	Exit test (do not move on until this passes)
Working CLI with trace and back-translation	All golden tests pass; errors exit non-zero without printing fake success

Phase 6 - Context resolver, ambiguity gate, suggestions (Week 6, first half)

Goal: Implement the headline novelty N1 plus N2 and N4.

> **Novelty work in this phase:** This is the core of the novelty claim. Prioritize it over any extra language features.

Steps

1. context.py (N1): before scoring, resolve (a) bare words against the symbol table (identifier vs string), (b) pronouns 'it/that/the variable' via the reference stack, (c) undeclared assignment -> implicit LET with warning, (d) near-miss names by edit distance <= 2 -> confirmation question, (e) type hints for scoring.
2. Re-score candidates using context: e.g. a candidate that uses an undeclared variable loses score; one that matches declared types gains.
3. ambiguity.py (N2): accept / clarify / reject using a score-margin threshold and missing-slot checks. Generate targeted questions from templates ('By how much should x increase?', 'Did you mean the variable score?').
4. Wire clarification into the REPL: user answer is merged into the slots and the candidate re-enters the pipeline. In file mode emit a diagnostic instead.
5. suggest.py (N4): for unsupported input, rank pattern templates by token overlap + edit distance; show the best example phrasing.
6. Tests: every ambiguous fixture triggers the expected question; context fixtures ('add 5 to it') resolve correctly; a flag `--no-context` disables N1 for the ablation.

Deliverable	Exit test (do not move on until this passes)
context.py, ambiguity.py, suggest.py; REPL clarification loop	All ambiguous and context fixtures pass; --no-context switch works

Phase 7 - Layered verifier (Week 6 second half - Week 7)

Goal: Implement V2 and V3 so every accepted statement is checked three ways.

> **Novelty work in this phase:** V3 is what upgrades 'round-trip' from a self-consistency check to a meaning check.

Steps

1. roundtrip.py (V2): parse generated MiniLang with the Phase 3 parser; compare AST to the original with dataclass equality; on mismatch report the first differing node.
2. ast_eval.py: evaluate the AST directly (variables dict + output list).
3. interpreter.py: execute MiniLang by parsing it and walking the parsed tree. Keep it separate from ast_eval.py (different files, ideally written by a different team member) so they do not share bugs.
4. V3: run both on the whole session program; compare final variable state and printed output. Cap REPEAT iterations for safety.
5. Verification summary in the trace: V1 sentence, V2 pass/fail, V3 pass/fail.
6. Mutation tests (test-only hook, never in the normal pipeline): swap operators, drop statements, change literals, alter precedence in generated code; check that V2/V3 flag them.

Deliverable	Exit test (do not move on until this passes)
verify/ package; verification shown in trace; mutation tests	All valid fixtures pass V2 and V3; mutation suite reports which mutations each layer catches

Phase 8 - Evaluation and ablation (Week 7)

Goal: Turn features into measured evidence (N5).

Steps

1. Build the evaluation set (separate from the tuning fixtures): see Section 8 for size and composition. Have one teammate write paraphrases without looking at the pattern table, to avoid testing only what you built for.
2. run_eval.py computes all Section 8 metrics and writes a CSV + summary table.
3. Ablation: run with context ON vs OFF (N1), and clarification ON vs OFF (forced top-1) (N2).
4. Record failures honestly; categorize them (unsupported phrasing, wrong slot, wrong resolution).
5. Fix only clear bugs found; do not tune on the evaluation set, or the numbers stop meaning anything.

Deliverable	Exit test (do not move on until this passes)
Results table, ablation table, failure analysis	run_eval.py reproduces the same numbers from a clean clone

Phase 9 - Hardening, documentation, report, demo (Week 8)

Goal: A reproducible, well-explained submission.

Steps

1. Regression test for every bug found; no crash on any fixture or random sentence.
2. Make README, LANGUAGE_SPEC, ARCHITECTURE, TESTING, NOVELTY match the actual implementation.
3. Write the report (Section 9 structure) with the evaluation tables.
4. Prepare the demo script (Section 7) and rehearse it; keep a recorded backup.
5. Optional only if all else is done: a minimal GUI or a Python backend.

Deliverable	Exit test (do not move on until this passes)
Final repo, report, demo script, recording	Another person reproduces install, tests, eval and demo from README alone

7. What the demo should show

Example trace for a multi-sentence session (default mode: back-translation always shown, confirmation only on CLARIFY). This is the single most persuasive slide for the novelty marks.

```
> make a variable called score and set it to 10
  intent: declare (rule D1, score 0.95)        decision: ACCEPT
  I understood: create variable score with value 10.
  MiniLang:  LET score = 10;          V2: pass   V3: pass

> add 5 to it                                    [N1 pronoun -> score]
  candidates: increment(score,5) 0.91           decision: ACCEPT
  I understood: increase score by 5.
  MiniLang:  score = score + 5;       V2: pass   V3: pass  (score = 15)

> print total                                    [N1: 'total' not declared]
  policy: not a declared name, not a typo -> text   decision: ACCEPT
  I understood: print the text "total".
  MiniLang:  PRINT "total";          V2: pass   V3: pass

> print scroe                                    [N1 typo check]
  decision: CLARIFY -> "Did you mean the variable score?"   user: yes
  I understood: print the variable score.   (confirmed)
  MiniLang:  PRINT score;

> increase score                                 [N2 missing slot]
  decision: CLARIFY -> "By how much should score increase?"   user: 2
  MiniLang:  score = score + 2;

> store 50 in bonus                              [N4]
  decision: REJECT -> unsupported. Try: "set bonus to 50"
```

Demo case	Shows
Valid declaration + pronoun reference	N1 context, V1 back-translation, V2/V3 pass
Variable vs text, typo confirmation	N1 symbol-table-informed interpretation, consistent with the Section 4 policy
Missing value, 'x to 10 or 20'	N2 clarification instead of guessing
Unsupported sentence	N4 suggestion, safe rejection
Semantic error (undeclared use in arithmetic)	Classical compiler diagnostics
Mutation test run	V2/V3 catch injected generation bugs (test hook, clearly labelled)
Ablation table	Measured value of N1/N2

8. Evaluation plan (N5)

Target dataset of about 150 English inputs, kept separate from development fixtures. Report actual numbers; do not promise percentages in advance.

Subset	Size	Content
Valid single statements	~50	All intents, canonical phrasing
Paraphrase groups	~15 groups x 3-4	Same meaning, different wording, written blind by a teammate
Multi-sentence sessions	~10 sessions	Pronouns, variable-vs-string, implicit declaration
Ambiguous	~25	Missing slots, alternatives, operator scope, name overlap
Invalid / unsupported	~25	Out-of-scope requests, malformed input, semantic errors

Metric	Definition	Supports
Exact-match accuracy	Generated MiniLang equals expected, on valid + session subsets	Core translation
Paraphrase consistency	Share of paraphrase groups where all members produce the same AST	Robustness of NLP
Ambiguity precision / recall	Of clarifications asked, how many were truly ambiguous / of truly ambiguous inputs, how many were caught	N2
Unnecessary-question rate	Clarifications asked on clearly unambiguous input	N2 usability
Context resolution accuracy	Correct pronoun / variable-vs-string / typo handling in sessions	N1
Safe rejection rate	Unsupported inputs rejected without crash or wrong code	Reliability
Suggestion usefulness	Share of rejected inputs where the top suggestion matches intended meaning (manually judged)	N4
Mutation detection by layer	Fraction of injected generation bugs caught by V2, V3, and combined	N3
Ablation deltas	Accuracy with N1 off vs on; wrong-code rate with N2 off (forced guess) vs on	Proof that N1/N2 matter

> **Honesty rule for the report:** show failures too. A results table with a few clearly explained failures is more credible than 100% everywhere, which examiners tend to read as testing only on examples you designed for.

9. Novelty statement for the report

Novelty and Original Contribution

Translating natural language into code and into domain-specific languages is an established area. This project does not claim to be the first such system. Its contribution is a **context-aware, self-verifying** translation pipeline for a formally specified mini-language, built on three ideas.

First, compiler-informed interpretation. Information flows back from the compiler to the language-understanding stage: the symbol table and statement history are used to resolve references such as 'it', to decide whether a word denotes a declared variable or literal text, to detect likely misspelled identifiers, and to apply type constraints when ranking interpretations.

Second, explicit ambiguity handling. The front end produces ranked candidate interpretations rather than a single guess, and a decision rule accepts, asks a targeted clarification question, or rejects with a suggested supported phrasing.

Third, layered verification. Each accepted statement is back-translated into canonical English so the user can confirm the interpretation, re-parsed to check structural consistency with the intended syntax tree, and executed by two independent evaluators to check behavioural equivalence.

The contribution is evaluated on a held-out set of valid, paraphrased, multi-sentence, ambiguous and unsupported inputs, including an ablation that measures the effect of compiler-informed interpretation and clarification, and mutation tests that measure which verification layer detects which class of generation error.

Likely viva questions and honest answers

Question	Answer
Isn't this just NL2Code, which already exists?	Yes, the area exists; we say so. Our claim is the compiler-to-NLP feedback, ambiguity handling and layered verification in a formally specified language, with ablation results.
When would round-trip ever fail?	Only on codegen/parser bugs, which is why it is one supporting layer. Back-translation and execution equivalence address interpretation and meaning.
Why not use an LLM?	Determinism, reproducibility and testability. An LLM front end could plug into the same candidate interface later, and the verifier would then matter more.
How do you know the NLP understood correctly?	We can't prove it in general; back-translation lets the user confirm, clarification avoids guessing, and the evaluation measures error rates.
What's the limitation?	Constrained English only; pattern coverage limits paraphrase handling; no functions, lists or user input in MiniLang.

10. Risks, coding-agent rules, definition of done

Risk	Mitigation
Novelty slips because core takes longer	Phase 6 has priority over extra grammar. If behind, drop REPEAT and ELSE before dropping N1/N2.
Too many clarification questions annoy users	Measure unnecessary-question rate; tune score margin; auto-resolve when symbol table decides it.
ast_eval and interpreter share a bug (V3 blind spot)	Separate files and authors; interpreter works on parsed MiniLang, not the original AST.
Evaluation set leaks into tuning	Write it after Phase 7, by a teammate, and freeze it before running.
Infinite loop in REPEAT during V3	REPEAT count must be a literal; cap at e.g. 1000; timeout guard.
Coding agent changes spec silently	CLAUDE.md rules below; review diffs to LANGUAGE_SPEC.md manually.
Deadline shorter than 8 weeks (not stated)	Confirm the date now. Compress Phases 3-5; keep 6-8.

CLAUDE.md (permanent rules for the coding agent)

```
1. Read README, DEVELOPMENT, LANGUAGE_SPEC, ARCHITECTURE and NOVELTY
   before any change.
2. Implement only the requested phase/step; list every changed file.
3. Keep module boundaries: NLP never emits MiniLang text;
   only semantic.py writes the symbol table.
4. Never expand grammar or English patterns without updating
   LANGUAGE_SPEC.md and tests together.
5. Keep ast_eval.py and interpreter.py independent; do not share evaluation code.
6. Mutation/corruption hooks exist only under tests/; never in the runtime pipeline.
7. Standard library only, plus pytest. No APIs, secrets or heavy
   dependencies without approval.
8. Add a test for every new behaviour and bug fix. Run tests;
   report the exact command and output.
9. Never claim tests passed unless they were run.
```

Definition of done

Requirement	Acceptance target
Core translation	At least 6 intents (declare, assign, inc/dec, print, if, repeat) with documented English patterns
Compiler concepts	Lexer, recursive-descent parser, typed AST, symbol table, semantic diagnostics, AST-based codegen
N1 context	Pronouns, variable-vs-string, implicit declaration, typo confirmation, with --no-context ablation switch
N2 ambiguity	Ranked candidates, accept/clarify/reject, REPL clarification loop
N3 verification	Back-translation, re-parse comparison, execution equivalence, mutation test results
N4 suggestions	Nearest supported phrasing for rejected input
N5 evaluation	Held-out set, all Section 8 metrics, ablation table, failure analysis
Reproducibility	Clean clone -> install -> tests -> eval -> demo, from README only

Final recommendation

Build the plain compiler pipeline first (Phases 0-5), but design the NLP output and session state from day one so the novelty features slot in without rework. Lead your novelty pitch with **compiler-informed interpretation** and **layered verification**, back both with the ablation numbers, and present structural round-trip as a supporting check rather than the headline.